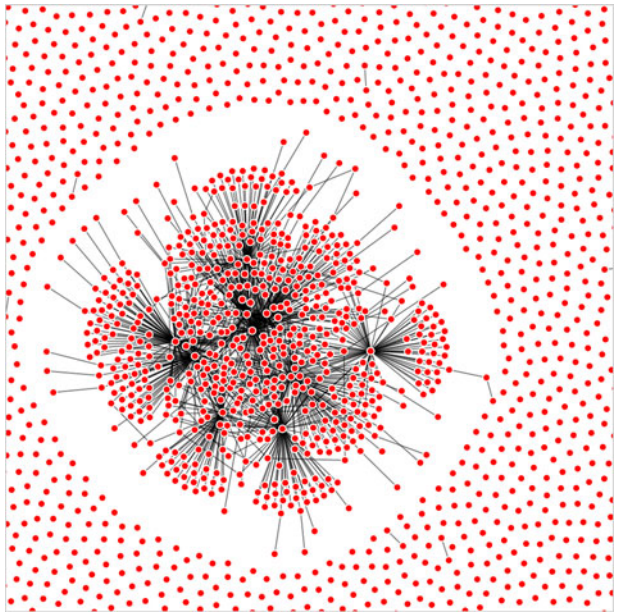


Figure 11.9 A randomly generated network with 3000 nodes and 1000 edges

shown in figure 11.9. What's lost in the static image is the slow and jerky rendering, even on a fast computer using a browser that handles SVG well.

When I first started working with these networks, I thought the main cause of slowdown was calculating the myriad positions for each node on every tick. After all, node position is based on a simulation of competing forces caused by nodes pushing and edges pulling, and something like this, with thousands of components, seems heavy duty. That's not what's taxing the browser, though. Instead, it's the management of so many DOM elements. You can get rid of many of those DOM elements by replacing the SVG lines with canvas lines. Let's change our code so that it doesn't create any SVG `<line>` elements for the links and instead modify our `forceTick` function to draw those links with canvas.

Listing 11.12 Mixed rendering network drawing

```
function forceTick() {
  var context = d3.select("canvas").node()
            .getContext("2d");
  context.clearRect(0,0,500,500);

  context.lineWidth = 1;
  context.strokeStyle = "rgba(0, 0, 0, 0.5)";

  sampleLinks.forEach(function (link) {
     context.beginPath();
     context.moveTo(link.source.x,link.source.y)
     context.lineTo(link.target.x,link.target.y)
     context.stroke();
  });
```

Remember: you always need to clear your canvas.

Draws links as 50% transparent black

Starts each line at the link source coordinates

Draws each link to the link target coordinates